Reviewer Pack — Minimal Rank of Tropical Differentials vs AC0 Parity Circuit…

Entry 9990a30cf746 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 05:12:11 UTC

Minimal Rank of Tropical Differentials vs AC0 Parity Circuit Size

Entry ID: `9990a30cf746` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 05:12:11 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): AC0 Parity Complexity

Statement:

> ['For any AC0 parity circuit C with n inputs, the minimal rank of its associated tropical differential form is Ω(2^(n/3)).', 'This holds true for all circuits computing the parity function on n variables.', 'The statement is invariant under the action of the symmetric group on the input wires.']

Rationale (proposer's reasoning):

> ['Tropical geometry provides a framework to study discrete structures through piecewise functions, which could potentially reveal deeper structures in complexity theory.', 'Differential forms are fundamental objects in tropical geometry that may capture essential properties of AC0 parity circuits not accessible by simpler tools.', 'A lower bound on the rank of tropical differentials for parity circuits would suggest a novel method to understand circuit complexity.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9b317c785671d80d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 80% of the generated AC0 parity circuits, the minimal rank of the associated tropical differential form is at least Ω(2^(n/3)) with a mean rank across all seeds not exceeding 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropical geometry' AND 'AC0 parity complexity' AND 'minimal rank' - 'tropical differential form' AND 'AC0 circuit size' AND 'Ω(2^(n/3))' - 'symmetric group action' AND 'parity function' AND 'tropical geometry'

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/math/0608534v2] On Automorphisms of Finite p-groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.7s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_tropical_differential(circuit):
        n = len(circuit)
        diff_form = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n):
            for j in range(i, n):
                if circuit[i] != circuit[j]:
                    diff_form[i][j] = 1
                    diff_form[j][i] = 1
        return diff_form

    def min_rank(diff_form):
        n = len(diff_form)
        rank = 0
        for i in range(n):
```

```
            row_sum = sum(diff_form[i])
            if row_sum > 0:
                rank += 1
        return rank

    def grothendieck_witt_class_mod_2(rank, n):
        return rank % 2 == 0

    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):
            circuit = generate_ac0_circuit(n)
            diff_form = compute_tropical_differential(circuit)
            rank = min_rank(diff_form)
            if not grothendieck_witt_class_mod_2(rank, n):
                return {
                    "metric_name": "min_rank",
                    "metric_value": rank,
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
                    "counterexample": f"n={n}, rank={rank}"
                }
            total_rank += rank
            instances_tested += 1

    mean_rank = total_rank / instances_tested
    return {
        "metric_name": "min_rank",
        "metric_value": mean_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank:.2f} std=... support_fraction={support_fraction:
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11292
2	preregistration	ollama_remote	glm4:latest	0	0	5357
3	novelty	ollama_remote	glm4:latest	0	0	4845
4	novelty	ollama_remote	glm4:latest	0	0	5831
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12196
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8293
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17542
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14185
9	judge	ollama_remote	glm4:latest	0	0	52180

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 131720 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9990a30cf746.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9990a30cf746.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9990a30cf746.tar.gz` (if generated)